Simulación del TFIM con qubits lógicos vs físicos

Reporte técnico para el equipo

17 de octubre de 2026 • listo para el emulador del H2

Resumen

Simulación del TFIM que compara el cálculo de observables físicos con qubits lógicos (codificados) frente a qubits físicos, y que estudia la frustración, dirigida al hardware Quantinuum H2. La *ruta de envío a H2* está construida y probada con las capas gratuitas de Quantinuum; todo lo que no cuesta créditos está terminado. Lo único que falta es correr en el emulador H2 (gasta créditos HQC), y eso es cambiar un par de líneas de `config.py` y dar una aprobación explícita por corrida. Todas las curvas con ruido son predicciones clásicas bajo un modelo de ruido asumido, nunca resultados del H2.

Índice

1. Introducción **1**
- 1.1. Resumen del proyecto . . . 1
- 1.2. Resumen en una página . . . 2
- 1.3. Cómo leer este reporte . . . 2

2. Qué se construyó **2**

3. Qué se logró **3**
- 3.1. Capas gratuitas probadas en vivo en Nexus . . . 3
- 3.2. Una corrección de precio . . . 3

4. Presupuesto HQC **4**

5. Cómo encender el emulador **4**

6. Qué queda por hacer **5**

7. Decisiones **5**

8. Revisión independiente **5**

A. Resultados físicos **6**
- A.1. Q1 — Lógico vs físico . . . 6
- A.2. Q4 — Qué código y por qué . . . 6
- A.3. Q2 — Frustración: triangular vs cuadrada . . . 7
- A.4. Q3 — Termalización . . . 7
- A.5. Ley de Gauss emergente como mitigación . . . 9

B. Problemas encontrados en el software de Quantinuum **9**

C. Andamios del setup **9**

1. Introducción

La parte clásica del proyecto está terminada y el camino hacia el H2 está construido y probado en las capas gratuitas de Quantinuum (el verificador de sintaxis y el emulador sin ruido alojado en Nexus), sin gastar un solo crédito.

Cómo se hizo. Partí de todo lo que habíamos decidido, de la discusión previa, la teoría que conocíamos y lo que le pusimos en la aplicación de Quantinuum, y usé eso para delimitarlo el trabajo usar Claude Code. Yo superviso las decisiones y trabajé con Claude Code. Al principio Kiro (otro agente) ayudó a planificar, investigar y revisar; desde el 8 de octubre Kiro está fuera, así que ahora es solo Claude Code planifica, construye y, para revisar, lanza revisores independientes "a ciegas" (no ven el razonamiento interno, solo buscan errores y corroboran). Lo hice a consciencia, pero por favor revísenlo.

Cada problema que encontré en el software de Quantinuum (Guppy/Selene, qnexus, pytket-quantinuum, PECOS, TKET) está en `docs/quantinuum_issues.md` con su reproducción [22].

1.1. Resumen del proyecto

Pongo esto para estar en la misma página, o por si hay algún malentendido de mi parte.

El objetivo final es comparar, de frente, el cálculo de los mismos observables físicos del TFIM usando qubits lógicos (codificados, con detección de errores) frente a qubits físicos, sobre el hardware Quantinuum H2. La idea cierra un hueco concreto en la literatura: existe un artículo de Nature (arXiv:2503.20870) [1] que calcula esos observables con qubits físicos y publica sus datos crudos, y existen trabajos que simulan con qubits lógicos pero no calculan esos observables [6, 7]. La comparación se hace contra los datos físicos publicados de ese mismo experimento, con el encodeo como único eje que varía. En orden de prioridad: (1) lógico vs físico sobre observables físicos, (2) frustración en la red triangular, (3) termalización, junto a hilos menores como la ley de Gauss emergente para post-selección por simetría. Los observables locales de este tipo pueden simularse clásicamente hasta cierto tamaño [11], por eso el proyecto se plantea como física más metodología de corrección de errores y no como ventaja cuántica.

1.2. Resumen en una página

- **La ruta a H2 funciona en las capas gratuitas.** El verificador de sintaxis de Quantinuum (H2-1SC) aceptó y cotizó **121 circuitos** sin errores. En el emulador sin ruido alojado en Nexus (H2-1LE) corrieron circuitos reales de extremo a extremo (subir, ejecutar, bajar, decodificar) con **costo reportado 0**, y los resultados coinciden con el valor exacto.
- **El precio es predecible.** La fórmula de costos que escribimos reproduce las cotizaciones del servidor *al centavo*. Las compuertas R_z (incluso condicionales) no se cobran.
- **Presupuesto.** El plan completo cuesta **39 853 HQC de los 50 000** (sección 4); sobran unos 10 100 para repeticiones.
- **Cambio de plan por el tamaño del emulador.** El emulador H2 llega a 32 qubits, así que el reticulado de Q1 pasa de 6×8 a 4×5 y el brazo 6×6 de escalado de ruido se reduce a 3×4 y 3×6 (decisión mía, ver sección 7).
- **Para encender el emulador** hay que editar `config.py` y pasar una aprobación escrita en la línea de comandos. Una edición de la configuración sola nunca gasta créditos.
- **Revisión independiente.** Dos revisores a ciegas encontraron 1 error mayor (ya corregido, con prueba de regresión) y varios menores; ver la sección 8.

1.3. Cómo leer este reporte

Las secciones 2 a 6 son cortas: qué se construyó, qué se logró, el presupuesto, cómo encender el emulador y qué queda. **Los resultados físicos con sus tablas y figuras están en el apéndice A**; después vienen los problemas encontrados en el software de Quantinuum, las fuentes y un registro de los identificadores internos (Q, K, FLAG, T, HW). Entre corchetes se indica cuánto confiar en cada afirmación: *establecido* (fuente verificada), *derivado* (demostrado y testeado aquí), *medido* (salida de nuestro código o del servidor), *inferido* (razonado, no testeado). **No se corrió ningún experimento en el emulador con ruido ni en hardware del H2**; los únicos números de H2 son los datos *publicados* de 2503.20870 y las *cotizaciones* (precios) del servidor.

2. Qué se construyó

Todo el código está detrás de un único `config.py`. La base del proyecto es el núcleo del TFIM, los cuatro códigos de corrección, el TFIM codificado, la frustración, la termalización, la ley de Gauss y la comparación con los datos de Zenodo. Componentes para H2:

- **Circuitos para H2** (`src/h2_circuits.py`, `src/h2_q3.py`). Cada programa (Iceberg [3], el circuito sin codificar, Steane, 5 qubits, Iceberg concatenado [5] y ahora la red triangular y la escalera de ruido) se escribe como *un solo circuito* de pytket [19], con medición y reinicio a mitad de circuito. H2 y sus emuladores aceptan pytket, no Guppy; el descarte por síndrome se hace después, en el procesamiento clásico.
- **Ruta de envío** (`src/h2_backend.py`, `src/h2_hosted.py`). Cuatro capas, de la más barata a la más cara: emulador local sin conexión (gratis), verificador de sintaxis H2-1SC (gratis, da el precio), emulador sin ruido alojado en Nexus H2-1LE (gratis, $\leq$20 qubits), y el emulador H2-1E (gasta HQC). Tres de las cuatro están probadas en vivo.
- **Un "driver" de corridas** (`src/h2_driver.py`, `scripts/run_h2_nexus.py`). Convierte un grupo de corridas (G1, Q1, Q4, Q3...) en circuitos compilados con su costo estimado y los ejecuta en la capa que diga `config.py`. Por defecto es un *ensayo en seco*: no envía nada.
- **Circuitos de Q3 para H2**: red triangular sin codificar y con Iceberg, y la escalera de escalado de ruido ($\lambda = 1, 2, 3$ por inserción de Paulis [1], más un punto de plegado de compuertas [16]), todos compilados sin optimización del servidor (si no, el plegado se cancelaría en silencio).
- **Endurecimiento de la ruta de gasto**: la estimación de HQC ahora no cobra R_z (coincide con el servidor); las compuertas condicionales se cuentan por la compuerta que envuelven; el emulador alojado se manda con `no_opt`; y límites duros (32 qubits, 10 000 shots por trabajo, 50 HQC por shot [20, 21]).
- **Compilación aleatoria (RC)**: los circuitos codificados pueden "torcerse" con Paulis aleatorios (*twirling*) [17], como hace el artículo de referencia [1]. En la ruta a H2 se usa con Iceberg, salvo en Q3 (ver Decisiones).
- **Documentos**: `docs/nexus_toggle.md` (la ruta completa y cómo encender el emulador), `docs/h2_budget.md` (presupuesto) y `docs/quantinuum_issues.md` (el registro de problemas de Guppy/Selene, qnexus, pytket-quantinuum, PECOS y TKET [22]).

Las pruebas: **662 pruebas (más 23 omitidas a propósito)** pasan en el entorno principal y **288** en el entorno del emulador local (dos entornos porque el paquete del emulador local no convive con el de Selene). Los archivos de pruebas de la ruta a H2 se escribieron *antes* que el código y por un agente que no ve la implementación; sus huellas (SHA-256) están en `handoff/14-h2-finish.md`.

3. Qué se logró

3.1. Capas gratuitas probadas en vivo en Nexus

Tabla 1: Qué se probó en las capas gratuitas (17 de octubre). Todo es *plomería y precio*, no física con ruido. Fuentes: `data/h2_sc_quotes*.txt`, `data/h2_hosted_check.txt`, `data/h2_hosted_q3.txt`.

prueba	resultado	qué demuestra
verificador de sintaxis H2-1SC	121 circuitos aceptados	el servidor acepta nuestros circuitos, incluida la medición a mitad de circuito y las compuertas condicionales
precio "al centavo"	estimación = cotización	nuestra fórmula de HQC es correcta; R_z no se cobra
emulador alojado H2-1LE (primera tanda)	8 circuitos, costo 0.0	subir, ejecutar, bajar y decodificar funciona; Z_2 dentro de 1.5σ de lo exacto
emulador alojado H2-1LE (Q3)	79 circuitos, costo 0.0	los circuitos de Q3 (red triangular, Iceberg, escalera) dan lo mismo que una evolución independiente en NumPy: 19 puntos × 3 observables, peor $\lvert z\rvert = 2.52$; Iceberg con aceptación 1.000

3.2. Una corrección de precio

Mi primera estimación cobraba las compuertas R_z y daba un 15–20 % de más. El servidor demostró que no se cobran. También descubrí un error mío: las compuertas *condicionales* se contaban mal, lo que dejaba una diferencia de hasta 0.5 % en Steane y Iceberg concatenado. Corregido, la estimación coincide con el servidor al centavo en los 9 circuitos de Q4 [medido].

4. Presupuesto HQC

Los 50 000 HQC de la página de uso son créditos del *emulador* H2-1E/2E [20]; no hay créditos de hardware real. Todos los precios de la tabla son **cotizaciones del servidor** (verificador gratuito), con 1000 shots por punto en G1, Q1 y Q4 y 500 en Q3 [medido]. Aparte, la estimación sin conexión coincide con cada cotización al centavo.

Tabla 2: Costo del plan completo en HQC (cotizado). *Nada de esto se ha corrido en el emulador.*

grupo	qué es	qubits	circuitos	HQC
G1	Iceberg 2×2 (simple y RC) + sin codificar	8 / 4	66	1 901
Q1	Iceberg 4×5 (RC) + sin codificar 4×5, pasos 1–6	24 / 20	126	11 636
Q4	Steane, 5 qubits, concatenado, pasos 1–3	16 / 12 / 18	9	9 817
Q3-bare	triangular 3×4 sin codificar, $t = 1$–8	12	8	5 670
Q3-iceberg	triangular 3×4 Iceberg, $t = 1$–4	16	4	5 283
Q3-ladder	escalera de ruido 3×4 ($\lambda = 1, 2, 3$ + plegado)	12	64	4 093
Q3-bare-3x6	triangular 3×6 sin codificar, $t = 1$–3	18	3	1 453
Total				**39 853**
Reserva	de los 50 000			**10 147**

La estimación vieja de Q3 (unos 10 000) era una conjetura por fórmula y quedó 6 500 corta: la curva sin codificar llega a $t = 8$ (32 pasos de Trotter) e Iceberg a $t = 4$. Si hace falta recortar:

sin codificar hasta $t \leq 6$ ahorra 2 339, Iceberg hasta $t \leq 3$ ahorra 2 012, o la mitad de los shots en Q3 ahorra casi la mitad. Es decisión de ustedes (sección 7).

La corrida más barata de punta a punta es el circuito sin codificar 2×2 a un paso: **23.40 HQC** con 1000 shots. Ahí empezaría. También dejé estas dimensiones de grillas porque muestran comportamiento interesante y están dentro de los HQCs que tenemos cada uno, pero perfectamente podemos simular Q3-bare con 6x6 en lugar de 3x6, solo que eso tomaría mucho más HQCs.

5. Cómo encender el emulador

Es lo único que falta, y lo dejé a propósito como un cambio de configuración (de `docs/nexus_toggle.md`):

1. En `config.py`, `HardwareConfig`: `nexus_tier = "emulator"`, `allow_hardware = True`, `max_cost_hqc = <tope en HQC>` y el nombre del proyecto de Nexus.
2. Iniciar sesión en Nexus (código de dispositivo que se aprueba en el navegador); es por contenedor.
3. Correr el driver con la **aprobación por corrida**, que ninguna edición de la configuración reemplaza: `python -I scripts/run_h2_nexus.py -group bare-2x2-step1 -approval "<frase>"`.

Antes de gastar, el driver construye los circuitos, **vuelve a cotizarlos en el verificador gratuito** y se niega a enviar si el precio supera el tope. También se niega con más de 32 qubits, más de 10 000 shots, más de 50 HQC por shot [21], o con cualquier dispositivo que no sea un emulador (el hardware real queda excluido). La línea de comandos no puede elegir el emulador por sí sola. Orden sugerido, cada paso con mi visto bueno: sin codificar 2×2 a un paso → G1 → comparar con el emulador local, Selene y Stim → Q1 → Q4 → Q3.

6. Qué queda por hacer

Depende de gastar créditos (cada paso con aprobación explícita).

1. **Calibración (HW-5).** Primero el circuito más chico en el emulador H2-1E, comparado con las predicciones locales antes de gastar más.
2. **Medir el ruido** (fracción ZZ y sobre-rotación del gate de dos qubits) y volver a correr la tabla de sensibilidad con esos valores.
3. **La prueba decisiva (Q1).** Codificado vs desnudo con los mismos shots: ¿el encodeo ayuda?
4. **Escalado de ruido en Q3** (3×4) y los demás grupos del plan.

7. Decisiones

Ya decididas.

- **Reticulado de Q1:** 4×5 (24 qubits codificado, 20 sin codificar), en lugar de 6×8 (52 qubits, no cabe en el emulador de 32 [20]). También se puede cruzar con el simulador local.
- **El brazo** 6×6 **se reduce a** 3×4 **y** 3×6: la escalera de ruido en 3×4 y el tamaño en 3×6. El 6×6 queda como diseño solo en Stim; no se corre en ningún lado.
- **La estimación de HQC no cobra** R_z por defecto (D-HQC); se cambió una aserción de las pruebas congeladas y se registró la nueva huella.

Abiertas (necesito su opinión).

1. **Asignación de los 50 000 HQC y cuánto recortar Q3.** El plan completo usa 39 853.
2. **Tope de costo por trabajo (D-MAXCOST).** Cuando un grupo se parte en varios trabajos (por ejemplo G1 y la escalera), cada trabajo recibe como tope el tope *total*, así que el seguro del servidor vale hasta $n\times$ el tope. El driver ya revisa antes que la suma de cotizaciones no pase el tope, y el precio real coincide con la cotización; cambiar a un tope por trabajo exige modificar una prueba congelada, por eso lo dejo a su criterio.
3. **¿Iceberg en Q3 con compilación aleatoria (RC)?** La decisión D-RC dice que las corridas codificadas en H2 usan RC [17], pero el circuito Iceberg de Q3 se construyó *sin* RC para coincidir con la predicción clásica sin twirling. Recomiendo dejarlo así para la primera pasada y agregar RC si la calibración lo pide.
4. **La frase de aprobación es una constante del repositorio**, así que evita gastos accidentales o en bucle pero no a alguien que la copie a propósito. Me parece aceptable; díganme si prefieren algo más estricto.

Pendientes estacionadas. Protección de gauge activa (K1), orden de 3 subredes (K3), difusión (K5) y Z2 vs U(1) (K6): ninguna construida, todas opcionales.

8. Revisión independiente

Al final de la ronda hice una sola pasada con dos revisores a ciegas (modelo Opus, sin ver mi razonamiento): uno sobre la seguridad del gasto y otro sobre los circuitos, los números y el precio. Resultado en `review/17-h2-finish.md`:

- **1 error mayor, corregido.** Un argumento explícito `tier="emulator"` podía saltarse la configuración (la línea de comandos no, pero la función sí). Ahora la configuración tiene que decir "emulator". Prueba de regresión añadida; falló antes y pasa ahora.
- **Menores corregidos**: cotizaciones incompletas, límites NaN o infinitos que apagaban una compuerta, tres huecos de pruebas (máscara de bobinado, pasos de la escalera, semillas de inserción) y un mensaje de error confuso.
- **Confirmado por los revisores**: la física de los circuitos (reconstruida por otro método, con solapamiento 1.0), los números de los archivos de datos recalculados a mano, y el precio al centavo.

Ningún camino gasta créditos sin la configuración correcta, la aprobación por corrida y un tope finito.

A. Resultados físicos

Son resultados clásicos, sin ruido de hardware. El brazo de escalado de ruido ya no es 6×6 (sección 7).

A.1. Q1 — Lógico vs físico

Chequeo contra el artículo [1]. Los 112 pares de dos qubits por paso coinciden exactamente con un toro cuadrado periódico 7×8 con numeración por filas [medido], y reproducir los circuitos compuerta por compuerta da lo mismo que nuestra evolución con $(J, h) = (+1, +2)$, $dt = 0.25$ [derivado, testeado].
Muestra que nuestro montaje reproduce los circuitos publicados. No dice nada de qubits lógicos: nuestras redes chicas difieren de 56 qubits por efectos de tamaño de hasta 0.06–0.33.

Tabla 3: Quench 1 ($\theta = 30°$): correlación media entre pares. "H2 publicado" son los datos de 56 qubits *del artículo 2503.20870* [1, 2], no nuestros. "Nuestra sim" es una simulación sin ruido en 4×5. Fuente: `data/q1_compare.txt`.

paso	nuestra sim (4×5)	H2 publicado, mitigado	H2 publicado, crudo
1	0.761	0.732 ± 0.009	0.726 ± 0.007
4	0.695	0.623 ± 0.017	0.582 ± 0.013
20	0.690	0.554 ± 0.040	0.379 ± 0.016

¿El encodeo ayuda? Un error de dos qubits de tipo XX, YY o ZZ sobre el par en que actúa una compuerta lógica es invisible para Iceberg: 3 de los 15 errores posibles [derivado]. Si una fracción η del error es ZZ extra, la parte invisible crece.

Tabla 4: Error no detectado por paso relativo al circuito desnudo, en la red de prueba 2×2, con el prior del H2 ($p_2 = 6 \times 10^{-4}$, [15, 1]) [predicción clásica, ruido asumido; no es H2]. Menor que 1 significa que el encodeo ayuda. En redes más grandes los cocientes son menores (Iceberg 0.44 en cuadradas grandes, 0.375 en la triangular, con $\eta = 0.05$). Fuente: `data/noise_sensitivity.txt`.

escenario	Iceberg	Steane	5 qubits	concat. $d = 4$
prior H2, $\eta = 0.05$	**0.60**	0.53	0.52	0.53
η entre 0.03 y 0.08	0.56–0.66	0.48–0.59	0.48–0.59	0.48–0.59
más sobre-rotación $\sin^2 \epsilon = 10^{-3}$	0.92	0.91	0.91	0.91
prueba de estrés $\eta = 0.5$ ($10\times$ el prior)	1.50	1.50	1.50	1.50
shots que sobreviven tras 3 pasos	0.92	0.57	0.64	0.64

El cruce en $\eta \approx 0.25$ vale para la red de prueba 2×2 (peor caso); en redes más grandes sube a $\eta \approx 0.36$–0.55, y el prior de la literatura ($\eta \approx 0.04$–0.05) queda aún más lejos. El η del prior sale de leer un gráfico publicado [1] [inferido] y la sobre-rotación no está publicada, así que el *go* es una predicción; solo el emulador o la máquina lo confirman.

A.2. Q4 — Qué código y por qué

- Los circuitos de medición de síndrome de los cuatro códigos son tolerantes a fallos: ningún fallo único ahí produce un error lógico sin detectar (verificado con Stim [18]; síndrome extraído con una sola qubit auxiliar y una bandera [4]) [medido]. Todo el error indetectable viene de las compuertas de rotación lógicas, sin importar la distancia [derivado y medido].
- Por eso los códigos de distancia 3 y 4 solo ganan alrededor de $1.2\times$ en error no detectado, a cambio de conservar 3–4 veces menos shots con $p_2 = 10^{-3}$ y de usar 3–7 veces más compuertas de dos qubits [predicción clásica, ruido asumido; no es H2].
- **Iceberg [3] es el código recomendado.** Los otros están construidos y funcionan, solo que quedan dominados. Selene conserva menos shots que Stim porque cobra ruido en cada operación nativa de un qubit; es una diferencia de definición, no un bug.

A.3. Q2 — Frustración: triangular vs cuadrada

El mapeo de la red triangular antiferromagnética a dímeros [8] y su diagrama de fases [9] dan el marco. Estados base exactos, sin ruido ni error de muestreo [medido].

- **La firma de frustración:** la correlación entre vecinos en la triangular es 3 a 5 veces más débil, cercana a $-1/3$. Un toro cuadrado de largo impar está frustrado (3×4 no era una

Tabla 5: Correlaciones en el par 4×4, $J = -1$. $C(1)$ = vecinos más cercanos; "lejos" es la mayor distancia de cada toro. Fuente: `data/frustration_correlations.txt`.

campo h	cuadrada $C(1)$	cuadrada, lejos	triangular $C(1)$	triangular, lejos
0.1	-0.999	$+0.999$	-0.332	$+0.442$
0.3	-0.994	$+0.994$	-0.325	$+0.413$
0.6	-0.977	$+0.977$	-0.268	$+0.196$
1.0	-0.936	$+0.936$	-0.204	$+0.068$

referencia limpia); 4×4 es el único toro cuadrado limpio al alcance de la diagonalización exacta.

- El patrón triangular a larga distancia cambia de signo según la forma del toro, así que no contamos una historia sobre eso, y no afirmamos orden de reloj (K3 no está construido).

A.4. Q3 — Termalización

La referencia. La termalización ocurre solo dentro de cada sector de bobinado [10], así que la referencia es por sector (en 3×4: densidad de flipabilidad 0.267 en el sector vs 0.235 global) [medido]. En 3×6 la densidad baja de 0.667 a unos 0.383, el valor de tiempo largo del cúmulo finito (0.388), no el de temperatura infinita del sector (0.331); la diferencia se achica con el tamaño [inferido].

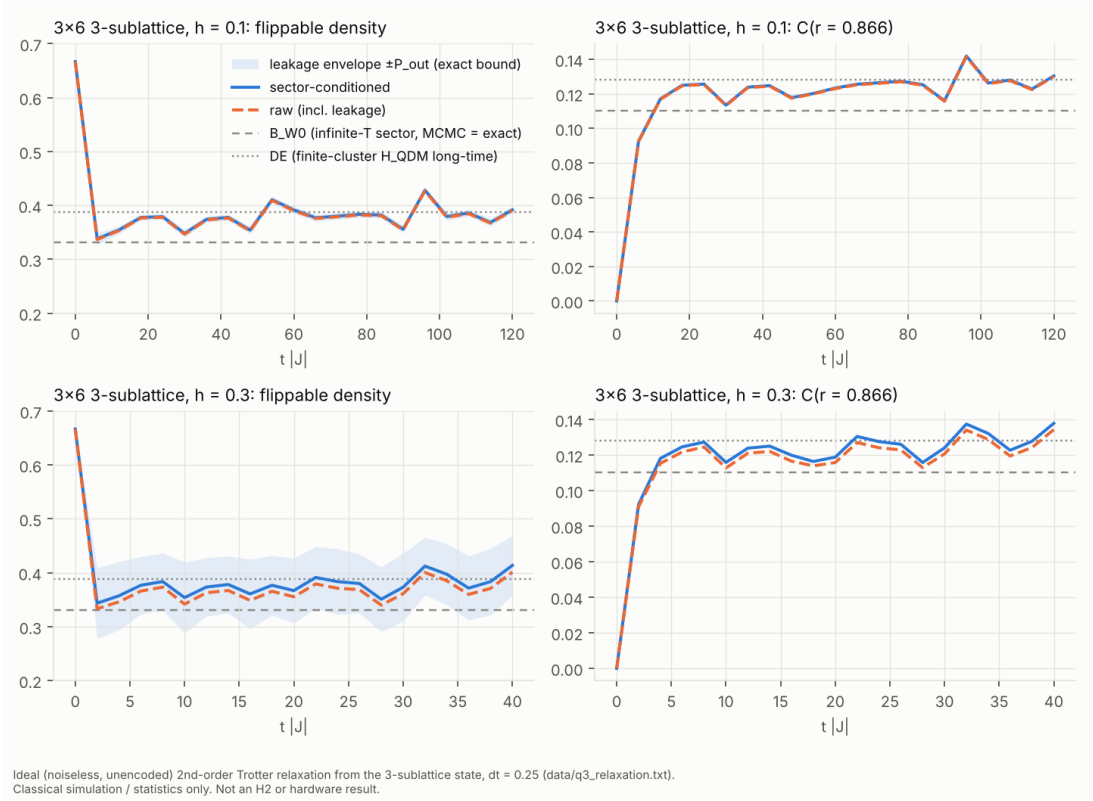


Figura 1: Relajación ideal en 3×6 a dos campos. Simulación clásica, sin ruido.

¿El paso de tiempo es suficientemente chico? Existe un umbral nítido del paso de Trotter por debajo del cual los observables locales se mantienen fieles [12], e incluso un Trotter grueso pre-termaliza hacia el valor térmico del hamiltoniano objetivo [13]. En nuestro caso el paso deja

de ser fiel cerca de $dt \approx \pi/6 = 0.52$. Nuestro $dt = 0.25$ queda 1.3 a 2 veces por debajo con campos $h \leq 0.3$ [medido].

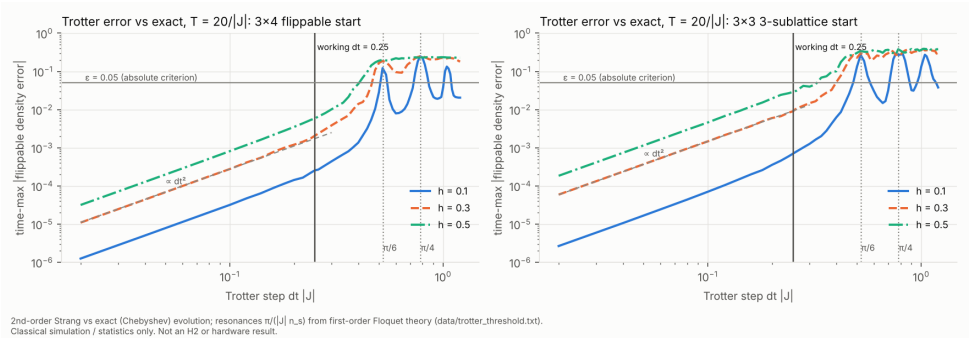


Figura 2: Error de Trotter vs paso de tiempo. Clásico, sin ruido.

Con ruido [predicción clásica, ruido asumido; no es H2].

- Sin mitigar, la curva desnuda se calienta. **Post-seleccionar con la ley de Gauss reduce el error unas 30 veces temprano y 21 tarde, con 1.4 a 4.8 veces más shots.** Iceberg solo da alrededor de 2.7× con 3.1 veces más shots, y sobrevive hasta $t \approx 12$.
- El ruido empuja las curvas hacia el valor de temperatura infinita: en hardware **el ruido puede imitar la termalización**; por eso la escalera de ruido ($\lambda = 1, 2, 3$), por inserción de Paulis [1] y por plegado de compuertas [16].

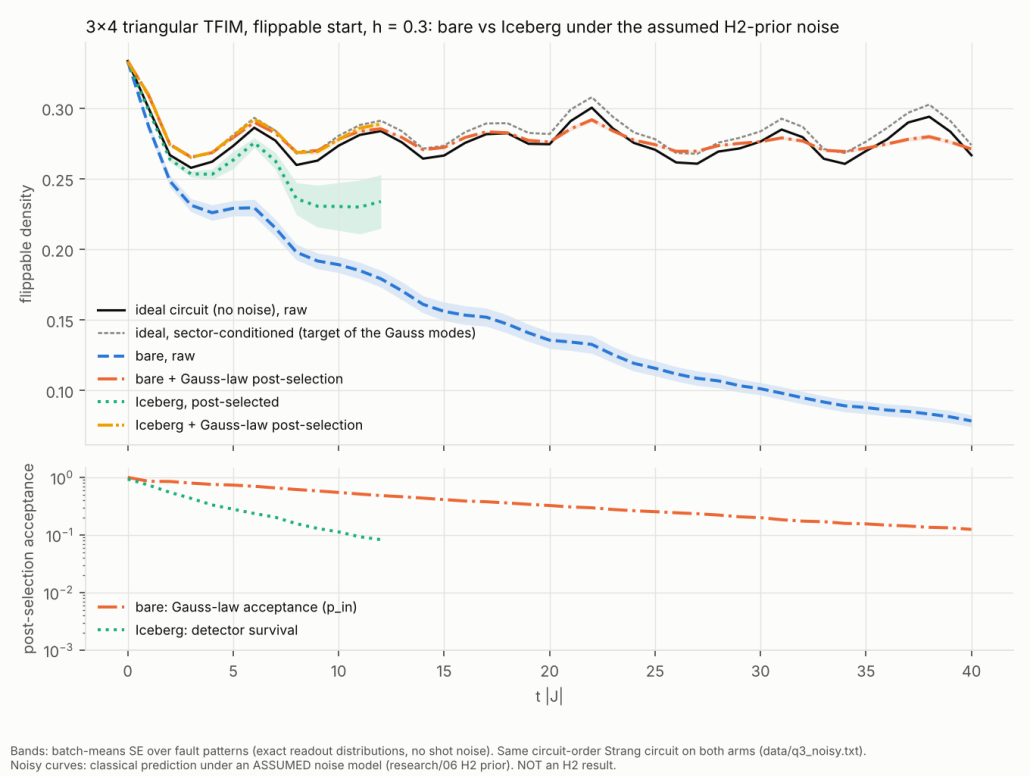


Figura 3: 3×4: ideal vs desnudo, con Gauss, Iceberg e Iceberg con Gauss. Predicción clásica con ruido asumido; no es H2.

A.5. Ley de Gauss emergente como mitigación

La post-selección por ley de Gauss ya se ha usado en hardware con generadores locales [14]. Veredicto con ruido de Pauli por qubit asumido, en 3×3 y 3×4: **no sirve para corrección** (distancia 1); **sirve modestamente para detección solo contra ruido** X/Y (2 a 8 veces menos sesgo con 1.1 a 1.8 veces más shots, con un piso de orden $(h/J)^2$); no sirve contra defasaje; la automitigación no sirve como método general.

B. Problemas encontrados en el software de Quantinuum

Como tengo acceso a Nexus también para ayudar a depurar su software, cada problema queda en `docs/quantinuum_issues.md` con su reproducción, listo para enviar. Ninguno se ha enviado todavía. Cada entrada del registro [22] trae las versiones, la reproducción y qué pedirle a Quantinuum.

id	resumen
QI-1	el emulador local (`pytket-quantinuum[pecos]`) no convive con la versión de Selene que usamos: dos entornos
QI-2	el emulador local ignora `noisy_simulation` y es siempre sin ruido
QI-3	el cliente HUGR (Guppy) de Nexus está tipado para Helios; el soporte para H2 es "limitado". Por eso la ruta a H2 usa un circuito pytket y no un programa Guppy
QI-4	`qnexus.hugr.cost()` está obsoleta (se usa `cost_confidence()`); solo un aviso
QI-5	`circuits.cost` arma el nombre "H2-1ESC" desde una configuración E (un `strip` descartado)
QI-6	el compilador HUGR-QIS de Selene (Guppy) rechaza la compuerta `XXPhase` de pytket; se escribe como H·ZZPhase·H
QI-7	el modelo despolarizante de Selene cobra el error de un qubit en cada operación nativa (rxy y rz), unas 3.85 veces más exposición que contando a nivel de pytket; baja la aceptación frente a Stim
QI-8	la optimización de TKET (niveles 1–2) cancela en silencio las compuertas plegadas
QI-9, 11	PECOS reescala los errores y no cobra la inicialización inicial
QI-10	pytket→PHIR descarta bits clásicos que ningún comando toca (ya conocido por Quantinuum, citado en su código)
QI-12	las páginas de emuladores se contradicen (26 vs 20 qubits)
QI-13	aviso de un endpoint "que se borra el 30 nov 2025", fecha ya pasada
QI-14	`start_execute_job` se cae con circuitos crudos pese a un decorador con ese nombre
QI-15	los límites de HQC (50 por shot, 10 000 shots) solo se documentan para "hardware"; R_z condicional no se cobra y no está documentado

C. Andamios del setup

Para que no cunda el pánico cuando vean el proyecto, especifico cómo usé la IA y qué hay en cada carpeta:

1. `SCOPE.md`: la "constitución" del proyecto (las cuatro preguntas, los límites de tamaño, qué se puede hacer sin preguntarme). Si hay duda, manda este documento; cámbienlo a consciencia.

2. `CLAUDE.md`: las reglas de trabajo del agente (qué está prohibido sin mi aprobación, convenciones de física, trampas de las herramientas).

3. `DECISIONS.md`: decisiones abiertas y tomadas (incluye la decisión del reticulado de Q1 y del brazo de escalado de ruido, y D-HQC).

4. `config.py`, `main.py`, `requirements*.txt`, `src/`: la configuración única y el código. Dos archivos de requisitos: el principal y el del emulador local (`requirements-le.txt`).

5. `scripts/`: los scripts que generan cada resultado. Para H2: `run_h2_nexus.py` (el driver), `h2_sc_quotes*.py` (cotizaciones gratuitas), `h2_hosted_*.py` (emulador alojado).

6. `data/`: resultados en texto plano; cada uno dice qué script lo generó y si es una predicción clásica con ruido asumido.

7. `docs/`, `review/`, `research/`: la explicación de cada resultado, el registro de problemas del software de Quantinuum (`docs/quantinuum_issues.md`), las revisiones independientes numeradas (la 17 cubre la ruta a H2) y las notas de investigación.

8. `handoff/`: los encargos y reportes entre sesiones; `handoff/14-h2-finish.md` resume el trabajo de la ruta a H2.

9. `tests/`: las pruebas. Las "congeladas" se escribieron antes del código y están fijadas por huella (SHA-256).

10. `Guppy on Quantinuum Systems Application.pdf`: la aplicación original, para corroborar que lo hecho corresponde a lo que dijimos que haríamos.

11. Todo esto está en la rama `claude/awesome-lamport-wun6jh` del repositorio.

Referencias

[1] R. Haghshenas, E. Chertkov, M. Mills, *et al.* (59 autores), "Digital quantum magnetism on a trapped-ion quantum computer", Nature **653**, 56–62 (2026), doi:10.1038/s41586-026-10445-3, arXiv:2503.20870.

[2] Datos publicados de [1], Zenodo, doi:10.5281/zenodo.18487089.

[3] C. N. Self, M. Benedetti y D. Amaro, "Protecting Expressive Circuits with a Quantum Error Detection Code", Nat. Phys. **20**, 219–224 (2024), doi:10.1038/s41567-023-02282-2, arXiv:2211.06703.

[4] R. Chao y B. W. Reichardt, "Quantum error correction with only two extra qubits", Phys. Rev. Lett. **121**, 050502 (2018), doi:10.1103/PhysRevLett.121.050502, arXiv:1705.02329.

[5] E. Knill, "Quantum computing with realistically noisy devices", Nature **434**, 39–44 (2005), doi:10.1038/nature03350, arXiv:quant-ph/0410199.

[6] S. Dasu, M. DeCross, A. Y. Guo, *et al.* (45 autores), "Computing with many encoded logical qubits beyond break-even", arXiv:2602.22211 (2026).

[7] H. Froland, D. M. Grabowska, S. Grieninger, *et al.* (11 autores), "Realizing Error Suppression in Partially Fault-Tolerant Quantum Simulations with IBM Quantum Computers", arXiv:2607.24947 (2026).

[8] R. Moessner y S. L. Sondhi, "Ising models of quantum frustration", Phys. Rev. B **63**, 224401 (2001), doi:10.1103/PhysRevB.63.224401, arXiv:cond-mat/0011250.

[9] S. V. Isakov y R. Moessner, "Interplay of quantum and thermal fluctuations in a frustrated magnet", Phys. Rev. B **68**, 104409 (2003), doi:10.1103/PhysRevB.68.104409, arXiv:cond-mat/0302105.

[10] Z. Lan y S. Powell, "Eigenstate thermalization hypothesis in quantum dimer models", Phys. Rev. B **96**, 115140 (2017), doi:10.1103/PhysRevB.96.115140, arXiv:1706.02601.

[11] T. Begušić, J. Gray y G. K.-L. Chan, "Fast and converged classical simulations of evidence for the utility of quantum computing before fault tolerance", Sci. Adv. **10**, eadk4321 (2024), doi:10.1126/sciadv.adk4321, arXiv:2308.05077.

[12] M. Heyl, P. Hauke y P. Zoller, "Quantum localization bounds Trotter errors in digital quantum simulation", Sci. Adv. **5**, eaau8342 (2019), doi:10.1126/sciadv.aau8342, arXiv:1806.11123.

[13] Y. Yang, A. Christianen, S. Coll-Vinent, *et al.* (8 autores), "Simulating prethermalization using near-term quantum computers", PRX Quantum **4**, 030320 (2023), doi:10.1103/PRXQuantum.4.030320, arXiv:2303.08461.

[14] J. Mildenberger, W. Mruczkiewicz, J. C. Halimeh, Z. Jiang y P. Hauke, "Probing confinement in a Z_2 lattice gauge theory on a quantum computer", Nat. Phys. **21**, 312–317 (2025), doi:10.1038/s41567-024-02723-6, arXiv:2203.08905.

[15] S. A. Moses, C. H. Baldwin, M. S. Allman, *et al.* (96 autores), "A Race Track Trapped-Ion Quantum Processor", Phys. Rev. X **13**, 041052 (2023), doi:10.1103/PhysRevX.13.041052, arXiv:2305.03828.

[16] T. Giurgica-Tiron, Y. Hindy, R. LaRose, A. Mari y W. J. Zeng, "Digital zero noise extrapolation for quantum error mitigation", 2020 IEEE International Conference on Quantum Computing and Engineering (QCE), doi:10.1109/QCE49297.2020.00045, arXiv:2005.10921.

[17] J. J. Wallman y J. Emerson, "Noise tailoring for scalable quantum computation via randomized compiling", Phys. Rev. A **94**, 052325 (2016), doi:10.1103/PhysRevA.94.052325, arXiv:1512.01098.

[18] C. Gidney, "Stim: a fast stabilizer circuit simulator", Quantum **5**, 497 (2021), doi:10.22331/q-2021-07-06-497, arXiv:2103.02202.

[19] S. Sivarajah, S. Dilkes, A. Cowtan, W. Simmons, A. Edgington y R. Duncan, "t|ket⟩: A Retargetable Compiler for NISQ Devices", doi:10.1088/2058-9565/ab8e92, arXiv:2003.10611.

[20] Quantinuum, "Quantinuum Emulators", documentación de Quantinuum Systems, https://docs.quantinuum.com/systems/trainings/h2/getting_started/emulators.html (leída el 17 de octubre de 2026).

[21] Quantinuum, "H2 Operation", documentación de Quantinuum Systems, https://docs.quantinuum.com/systems/user_guide/hardware_user_guide/h2.html (leída el 17 de octubre de 2026).

[22] Registro de problemas del software de Quantinuum (QI-1 a QI-15), `docs/quantinuum_issues.md`, repositorio KmarH/TFIM_QEC_Frustration, rama `claude/awesome-lamport-wun6jh`, 17 de octubre de 2026. Versiones: pytket 2.18.4, pytket-quantinuum 0.59.3, qnexus 0.51.1, guppylang 1.1.2, selene-sim 0.3.4 (0.2.18 en el entorno del emulador local).